

RAG System Architecture & Empirical Design Explanation

1. Design Parameter Chosen: Chunk Size & Overlap (`chunk_size=500` , `chunk_overlap=50`)

We empirically evaluated three text chunking configurations on technical documentation (1,200 words):

- **Config A (300/30):** Produced 42 chunks (avg 159.93 chars). Fragmented sentences mid-thought, degrading semantic retrieval recall.
- **Config B (500/50) [CHOSEN]:** Produced 19 chunks (avg 345.05 chars). Provided the optimal trade-off, preserving complete conceptual thoughts per chunk while maintaining a low vector index memory footprint.
- **Config C (800/80):** Produced 12 chunks (avg 538.83 chars). Broader chunks introduced background noise into vector similarity matching and inflated prompt token bloat (`2.67x`). *Decision:* `chunk_size=500` and `chunk_overlap=50` locked as system default.

2. Observed Failure During Building: Cosine Distance Metric Inversion

- **Failure Observed:** Initial query tests against ChromaDB returned raw values around `0.18` for highly relevant matches, causing the similarity threshold (`threshold=0.35`) to falsely reject relevant queries and return `"not found in the provided documents"`.
- **Root Cause:** ChromaDB's HNSW cosine distance space returns raw distance $d \in [0.0, 2.0]$, where smaller values indicate higher similarity ($d = 1.0 - \text{cosine_similarity}$). Comparing raw distance d directly against a similarity threshold created a logical inversion.
- **Remediation:** Implemented score normalization in `VectorStoreService`: $\text{Score} = \max(0.0, \min(1.0, 1.0 - d))$ in `app/services/vector_store.py` to guarantee normalized similarity in $[0.0, 1.0]$.

3. Metric Tracked: Per-Stage Latency Breakdown & Grounding Short-Circuiting

- **Metric:** Instrument microsecond latency metrics (`embedding_ms` , `retrieval_ms` , `generation_ms` , `total_ms`) in structured logs (`query_metric`).
- **Insights & Findings:** For grounded queries passing the similarity threshold (`score >= 0.35`), LLM generation (`gpt-4o-mini`) dominated overall latency ($\text{generation_ms} \approx 310\text{ms}$ vs $\text{retrieval_ms} \approx 85\text{ms}$, total $\approx 438\text{ms}$). When queries failed the threshold (unsupported/off-topic), the grounding guardrail short-circuited execution ($\text{generation_ms} = 0.00\text{ms}$), reducing end-to-end request latency from **438ms to 119ms** (a **73% latency reduction**) while completely eliminating LLM token costs.

4. Unfinished Work & Next Steps

- **Unfinished:** Synchronous PDF parsing and embedding during `POST /upload` blocks HTTP request threads during heavy file uploads.
- **Next Steps:**
 1. **Asynchronous Task Queue:** Offload PDF parsing, chunking, and vector embedding from HTTP request threads to background workers (e.g., Celery / TaskIQ backed by Redis) for non-blocking uploads.
 2. **Cross-Encoder Re-Ranking:** Implement a secondary re-ranking stage (e.g., `FlashRank` or `Cohere Rerank`) to re-score top 20 candidate chunks down to top 5 before prompt assembly.
 3. **Hybrid Search (BM25 + Dense Vectors):** Combine sparse BM25 keyword matching with dense vector embeddings via Reciprocal Rank Fusion (RRF) to excel at exact term and code identifier queries.